NIIT | StackRoute

Agentic AI Systems Engineering

Foundations · RAG · Multi-Agent Orchestration · Security · Evaluation

Two-Phase Bootcamp · 40 Hours · 5 Days × 8 Hours · Python Track

Target Audience	Senior IT, CS, EE & Mechanical Engineers (8–15 years experience) — Python Track
Format	In-person Bootcamp · Hands-on Labs · Guided Group Exercise
Delivery	Phase 1: 2 Days (16 Hours) · Phase 2: 3 Days (24 Hours)
Tools	OpenAI SDK · Anthropic Claude SDK · LangChain · LlamaIndex · LangGraph · MCP · Python
Version	v6.0 · August 2026 · Revised per Client Discussion (29-Jul-26)

Track Note: This design covers the Python-based agent development track only. Java + Spring AI is planned as a separate, parallel curriculum and is out of scope for this document, per client discussion.

Program Overview

Two-Phase Bootcamp · 40 Hours · 5 Days × 8 Hours · Revision v6.0

This revised programme delivers Agentic AI Systems Engineering in 40 hours across 5 days for the Python development track (Java/Spring AI now runs as a separate curriculum per client request). Phase 1 (2 days, 16 hours) establishes GenAI, NLP, OCR, embeddings, prompt engineering, and dual-SDK (OpenAI + Anthropic Claude) single-agent foundations. Phase 2 (3 days, 24 hours) advances into enterprise RAG pipelines, multi-agent orchestration and agentic design patterns with LangGraph, MCP-based enterprise integration, cloud deployment and LLMOps, security/access-control and AI governance, and evaluation with observability.

Programme Structure

Phase	Days	Key Focus Areas
1	2(16 hrs)	GenAI · NLP · Generic & Cloud OCR · LLMs · Transformers · Embeddings (incl. multilingual) · Prompt Engineering · OpenAI SDK · Claude SDK · Function/Tool Calling · LangChain · Agent Skills overview · Memory · Single-Agent Mini-Project
2	3(24 hrs)	Enterprise RAG (LangChain + LlamaIndex) · Fine-Tuning Overview & Decision Framework · Agentic Patterns (ReAct/Reflection/Plan-Execute) within Multi-Agent Orchestration · LangGraph · MCP Enterprise Integration · A2A · Memory Architectures · Cloud Deployment (Azure/AWS) & MLOps/LLMOps overview · Access Control & AI Governance · Guardrails · Observability (LangSmith/Langfuse) · Evals · Guided Group Exercise

Design Decisions — v6.0 Revision (Client Discussion, 06-Aug-26)

1. Python and Java tracks separated; Spring AI and Java-specific content moved out of this design and will be delivered as a standalone Java track.
2. RAG module expanded with additional enterprise use cases — IT knowledge base, policy/contract search, customer support, and engineering documentation search.
3. Dual SDK coverage added — OpenAI SDK and Anthropic Claude SDK — taught in parallel throughout Phase 1 and used interchangeably in Phase 2, since the team works with both model providers.
4. Agent development remains fully code-based throughout. No-code/low-code platforms — including Azure AI Foundry's low-code agent builder — are covered only as a brief awareness overview, not as a hands-on lab.
5. Cloud deployment overview added to Phase 2 — Azure AI Foundry/Azure OpenAI and AWS Bedrock, model deployment options, and MLOps/LLMOps fundamentals (overview only, no deep dive on either platform).
6. OCR coverage broadened — generic/open-source OCR (Tesseract) hands-on lab retained, plus an overview and output comparison against cloud OCR services (Azure AI Vision OCR, AWS Textract).
7. Overview of "Skills" added — reusable agent capability packaging (Claude Agent Skills pattern) — as an emerging pattern for structuring agent capabilities.
8. LlamaIndex added alongside LangChain as a second RAG framework, with a hands-on comparison of both.
9. MCP coverage extended to demonstrate connecting agents to enterprise data sources — SharePoint, Confluence, Jira, and internal databases.

General - RPS Data

10. Agentic design patterns (ReAct, Reflection, Plan-and-Execute) moved to follow RAG, and are now taught as part of Multi-Agent Orchestration with LangGraph rather than as a standalone pre-RAG module.
11. New security and data-safety content added — role-based, team-based, and geography-based access control for AI/LLM systems, alongside guardrails and AI governance frameworks.
12. Observability coverage expanded to include both LangSmith and Langfuse tracing.
13. Added a Fine-Tuning Overview with a concrete decision framework — fine-tuning vs RAG vs prompt engineering vs hybrid — based on data availability, latency, cost, and update frequency.
14. Prerequisite simplified to Python programming proficiency only; the Java prerequisite is removed along with Spring AI.
15. Lab environment and infrastructure requirements (cloud sandbox access, connector credentials) to be discussed and finalised separately with the client IT team.

Additional Enterprise Integration Coverage (Team Inputs)

- Deployment strategy overview covering both Azure and AWS, with Azure AI Foundry / Azure OpenAI Service and AWS Bedrock presented as the two reference deployment paths.
- Brief awareness-level coverage of Azure AI Foundry's low-code agent creation options, explicitly contrasted with the code-first approach used throughout this course.
- Sentence Transformers and multilingual embeddings included within the NLP/embeddings coverage in Module 1.
- Dynamic model and prompt selection/routing patterns using LangGraph included within the Multi-Agent Orchestration module.
- Enterprise document connectors — SharePoint, Confluence, Jira — via LlamaIndex Hub and Unstructured API included within the RAG module.
- OCR clarification: hands-on coverage uses generic/open-source OCR (Tesseract); Azure AI Vision OCR is covered as a cloud-service overview and output comparison, not a separate hands-on lab.

Pacing Risk Flag

Day 4 (Multi-Agent Orchestration, Agentic Patterns and MCP combined) is now the most content-dense day in the programme, since agentic patterns previously spread across two days are consolidated here alongside orchestration and MCP. Hands-on lab durations have been tightened accordingly; if live pacing slips, the A2A coordination demo and one pattern lab (Reflection or Plan-and-Execute) are the recommended trims, with the concept still covered via instructor walkthrough.

Learning Outcomes — Complete Programme

- Design and build single- and multi-agent AI systems using OpenAI SDK and Anthropic Claude SDK with LangChain.
- Apply NLP, multilingual embeddings, and OCR (open-source and cloud) in enterprise document intelligence workflows.
- Build production RAG pipelines with LangChain and LlamaIndex, including enterprise data-source connectors.
- Apply a fine-tuning decision framework to choose between fine-tuning, RAG, prompt engineering, and hybrid approaches.
- Implement agentic design patterns (ReAct, Reflection, Plan-and-Execute) within multi-agent orchestration using LangGraph.
- Integrate MCP servers to connect agents with enterprise systems (SharePoint, Confluence, Jira, databases) and design A2A coordination workflows.
- Understand cloud deployment options (Azure AI Foundry/Azure OpenAI, AWS Bedrock) and MLOps/LLMOps fundamentals.
- Engineer guardrails, role/team/geography-based access controls, and AI governance practices for production agents.
- Evaluate and observe agents using LangSmith/Langfuse tracing and task, groundedness, and safety evaluation frameworks.

Client Discussion — Traceability

Every discussion point mapped to where it is addressed in this design

#	Client Discussion Point	How It Is Addressed in v6.0
1	Separate Python/Java tracks; move out Spring AI	Spring AI and Java content removed; Track Note on title page; Java positioned as a separate curriculum
2	More enterprise use cases for RAG	Module 3: IT knowledge base, policy/contract search, customer support, engineering docs use-case walkthrough
3	Cover both OpenAI and Anthropic Claude SDKs	Module 1 (foundations) and Module 2 (parallel SDK labs); used interchangeably in Phase 2 builds
4	Coding-based agent development only; no-code/low-code out of scope	All hands-on labs are code-based; Azure AI Foundry low-code builder covered only as an awareness overview in Module 5
5	Azure AI overview, model deployments, MLOps/LLMOps in Phase 2 (no deep dive)	Module 5: Cloud Deployment & MLOps/LLMOps Overview (Azure AI Foundry/Azure OpenAI, AWS Bedrock)
6	Generic OCR + overview of Azure OCR or similar	Module 1: Tesseract OCR hands-on lab + Azure AI Vision OCR / AWS Textract overview and comparison
7	Overview of "Skills"	Module 2: Agent Skills Overview — reusable capability packaging
8	Cover LlamaIndex in RAG pipelines	Module 3: LlamaIndex covered alongside LangChain, with a hands-on comparison lab
9	Connect enterprise data sources via MCP	Module 4: MCP Enterprise Connector demo — SharePoint, Confluence, Jira, internal databases
10	Move agent patterns after RAG; cover under Multi-Agent Orchestration with LangGraph	Module 4 restructured: ReAct/Reflection/Plan-and-Execute taught within Multi-Agent Orchestration
11	Security & data safety — role/team/geography access, guardrails, AI governance	Module 5: Access Control & Data Safety, Guardrails Framework, AI Governance
12	More observability coverage — LangSmith and Langfuse	Module 5: Observability lab instrumenting agents with both LangSmith and Langfuse
13	Overview of fine-tuning and techniques	Module 3: Fine-Tuning Overview + Fine-Tuning Decision Framework and workshop
14	Prerequisite: Python programming knowledge only	Prerequisites page updated — Java prerequisite removed
15	Lab requirements to be discussed and finalised	Noted explicitly on the Prerequisites & Lab Environment page
16	Deployment strategy: Azure / AWS; Azure AI Foundry; Azure OpenAI low-code agents	Module 5: Cloud Deployment Overview + low-code awareness note
17	LlamaIndex	Module 3 (see above)
18	NLP with Sentence Transformers / multilingual embeddings	Module 1: Embeddings section extended

#	Client Discussion Point	How It Is Addressed in v6.0
19	Dynamic models/prompts using LangGraph	Module 4: Multi-Agent Orchestration lab — dynamic model & prompt routing
20	Azure AI Vision OCR vs generic OCR clarification	Module 1: Tesseract (hands-on) + Azure AI Vision OCR (overview/demo comparison)
21	Fine-Tuning Decision Framework	Module 3: dedicated framework + 4-scenario workshop
22	Enterprise Document Connectors (SharePoint, Confluence, Jira via LlamaIndex Hub / Unstructured API)	Module 3: Enterprise Connector Lab; also reinforced via MCP in Module 4

PHASE 1

Foundations of Agentic AI & Single-Agent Development · 16 Hours · 2 Days × 8 Hours · OpenAI SDK · Anthropic Claude SDK · LangChain · Python

Phase 1 establishes the knowledge and practical skills to build AI-powered applications and autonomous single agents, now covering both OpenAI and Anthropic Claude SDKs side by side. Participants move efficiently through GenAI, NLP, and OCR (generic + cloud overview), Transformer architecture, embeddings (including multilingual), Prompt Engineering, and dual-SDK development, finishing with a functional enterprise single-agent build in LangChain. Foundational concepts familiar to senior engineers (ML basics, Responsible AI) are treated concisely to protect hands-on time.

Phase 1 — Detailed Module Design

Module	Topics / Technical Coverage	Hrs	Hands-on	Tools
Module 1GenAI, NLP, OCR & LLM Fundamentals **6 Hours**	AI vs ML vs Deep Learning vs GenAI (survey-level — 8–15yr engineers) Foundation Models · Enterprise AI Landscape NLP Pipeline: tokenisation, POS tagging, NER, semantic similarity, intent detection OCR Fundamentals: traditional OCR vs AI-assisted OCR, document layout, enterprise document types Generic OCR (Tesseract) hands-on pipeline; Cloud OCR overview — Azure AI Vision OCR, AWS Textract (comparison, no separate lab) Transformer Architecture: attention mechanism, encoder-decoder models LLM Architecture: context windows, hallucinations and limitations Embeddings: vector representations, semantic similarity; Sentence Transformers; multilingual embeddings Model Selection Strategy: proprietary vs open, cost and capability fit — OpenAI and Anthropic Claude model families Responsible AI: key principles in 30 minutes — bias, transparency, governance (no separate exercise)	**6**	• Enterprise use-case mapping: teams match agent type to model family across 4 IT scenarios (30 min) • LLM behaviour exploration: hallucination triggers, context-window edge cases, temperature effects (45 min) • OCR lab: generic OCR (Tesseract) on invoice/support-ticket samples + cloud OCR (Azure AI Vision) output comparison demo (45 min) • Multilingual embeddings & semantic similarity mini-lab using Sentence Transformers (30 min) • Model selection exercise: cost vs capability decision matrix for a given enterprise scenario (15 min)	Python 3.12+ Jupyter / VS Code OpenAI API Anthropic Claude API Tesseract OCR Azure AI Vision OCR (demo) HuggingFace Transformers Sentence-Transformers Pre-built notebooks Model comparison sheet
Module 2Prompt Eng, OpenAI & Claude SDKs, LangChain & Single Agent	Prompt Engineering Fundamentals: zero-shot, few-shot, chain-of-thought, role and instruction prompting Structured Outputs: JSON schema enforcement, Pydantic models, output validation (OpenAI + Claude) Prompt Templates and Prompt Debugging	**10**	• Prompt optimisation lab: zero-shot vs few-shot vs CoT on IT support-ticket triage, tested on OpenAI and Claude (45 min) • Structured output generation: JSON schema	Python 3.12+ OpenAI SDK Anthropic Claude SDK LangChain (latest) LangSmith (tracing)

Module	Topics / Technical Coverage	Hrs	Hands-on	Tools
10 Hours	OpenAI SDK: API fundamentals, model selection, function calling, streaming responses Anthropic Claude SDK: Messages API, tool use, streaming responses — parallels to OpenAI SDK Agent Skills Overview: packaging reusable agent capabilities/instructions as Skills — concept and when to use Cost Optimisation: token budgeting, caching, batching (across providers) Observability Basics: logging, tracing, error handling OpenSpec Agent Design (30-min walkthrough): problem → spec → tool contracts → role boundaries → escalation rules LangChain Architecture: chains, agent executors, LCEL pipelines, output parsers (provider-agnostic) Tool Integration: search, API, database, code-execution tools Memory Types: conversation buffer, summary memory, window memory Context and Session Management · Conversational Workflow Design Agent Architecture Blueprint for enterprise workflows		enforcement on document extraction, OpenAI + Claude (30 min) ● Build AI assistant with OpenAI SDK: function calling + streaming (30 min) ● Rebuild the assistant with Anthropic Claude SDK: tool use + streaming — compare API design (30 min) ● Agent Skills walkthrough: convert a repeated prompting pattern into a reusable Skill (20 min) ● OpenSpec walkthrough: complete a partially-filled agent spec for an IT ops use case as a team (30 min) ● Build tool-enabled LangChain agent: web search + DB query + summarisation, provider-configurable (60 min) ● Memory comparison: buffer vs summary memory on multi-turn support scenario (40 min) ● Mini-Project — Enterprise Operations Assistant: incident triage → knowledge lookup → structured escalation output (60 min) ● Architecture review: identify context-window risks and bottlenecks (15 min)	Pydantic SerpAPI / Tavily SQLite sample DB OpenSpec template Agent trace sheet Jupyter / VS Code

PHASE 2

Enterprise RAG, Multi-Agent Orchestration, Cloud & Governance · 24 Hours · 3 Days × 8 Hours · LangGraph · LlamaIndex · MCP · Azure/AWS

Phase 2 builds directly on the single-agent, dual-SDK foundation from Phase 1. Participants implement enterprise-grade RAG pipelines (LangChain + LlamaIndex) with enterprise data connectors and a fine-tuning decision framework, then move into agentic design patterns taught within multi-agent orchestration using LangGraph, MCP-based enterprise integration, cloud deployment and MLOps/LLMOps awareness, security/access-control and AI governance, and evaluation with observability. Day 5 concludes with a structured guided group exercise that integrates all Phase 2 components in a team setting.

Phase 2 — Detailed Module Design

Module	Topics / Technical Coverage	Hrs	Hands-on	Tools
Module 3 Enterprise RAG Pipelines & Fine-Tuning Strategy **8 Hours**	RAG Fundamentals: retrieval-augmented generation pipeline, when to use vs fine-tuning vs prompt stuffing Fine-Tuning Overview: full fine-tuning, LoRA/PEFT, instruction tuning (overview level) Fine-Tuning Decision Framework: fine-tuning vs RAG vs prompt engineering vs hybrid, based on data availability, latency, cost and update frequency Vector Databases: FAISS, Chroma, Pinecone — architecture and trade-offs Chunking Strategies: fixed, semantic, hierarchical — comparison Metadata Design · Hybrid Retrieval: keyword + vector, re-ranking Retrieval Quality Evaluation: precision, recall, answer relevance RAG Frameworks: LangChain RAG vs LlamaIndex — architecture comparison, when to use each Enterprise Document Connectors: SharePoint, Confluence, Jira via LlamaIndex Hub / Unstructured API Enterprise RAG Use Cases: IT knowledge base, policy/contract search, customer support, engineering documentation Agentic RAG: agent-driven retrieval, tool-wrapped retrievers, knowledge management	**8**	• Build end-to-end RAG pipeline with LangChain: ingest → chunk → embed → retrieve → answer on support-doc corpus (45 min) • Rebuild the same pipeline with LlamaIndex: compare developer experience and output quality (30 min) • Chunking strategy comparison: fixed vs semantic on same document set (30 min) • Retrieval quality evaluation: measure precision and answer relevance on test queries (30 min) • Enterprise connector lab: connect the RAG pipeline to a sample SharePoint/Confluence space via LlamaIndex Hub (45 min) • Enterprise use-case walkthrough: build a policy-search assistant and a customer-support RAG assistant from a shared pattern (45 min) • Agentic RAG exercise: wrap the retriever as a tool for an agent to call selectively (25 min)	Python 3.12+ LangChain LlamaIndex OpenAI SDK Anthropic Claude SDK FAISS / ChromaDB LlamaIndex Hub connectors Unstructured API Jupyter / VS Code Chunking workbook Retrieval eval rubric Fine-tuning decision canvas

Module	Topics / Technical Coverage	Hrs	Hands-on	Tools
Module 4 **Multi-Agent Orchestration with LangGraph, Agentic Patterns & MCP** **8 Hours**	Agentic AI Architecture: single-agent vs multi-agent design decisions LangGraph Fundamentals: stateful graphs, nodes, edges, conditional routing Agentic Design Patterns within LangGraph: ReAct (Reason + Act) — thought-action-observation loop, failure recovery Reflection / Self-Correction Pattern: critique-and-revise loop, quality gates Plan-and-Execute Pattern: planner-executor split, dynamic replanning Pattern Selection Framework: reliability vs complexity, enterprise workflow fit Multi-Agent Orchestration: supervisor/orchestrator-worker topologies; dynamic model & prompt routing using LangGraph State Schema Design and Checkpointing MCP (Model Context Protocol): architecture, servers and clients, tool contracts, capability exposure MCP for Enterprise Data Sources: connecting agents to SharePoint, Confluence, Jira and internal databases via MCP servers A2A (Agent-to-Agent) Coordination: message passing, delegation, trust boundaries Advanced Memory Architecture: short-term/state, long-term/knowledge, episodic, procedural Context Engineering: memory selection strategy, summarisation, compression Human-in-the-Loop (HITL) Patterns: approval gates, interrupt-and-resume, override flows Event-Driven Agent Systems and Enterprise Integration Patterns	**8**	• ReAct pattern walkthrough within LangGraph on a knowledge-resolution scenario (25 min) • Reflection pattern comparison lab: with and without critique loop (25 min) • Plan-and-Execute lab on a multi-step IT ops workflow (30 min) • Pattern selection exercise: document failure modes and enterprise fit for each pattern (15 min) • Multi-agent orchestration lab: supervisor routes to specialist agents with dynamic model/prompt selection (45 min) • MCP integration: expose a sample enterprise API as an MCP server, connect agent client (35 min) • MCP enterprise connector demo: connect an agent to SharePoint/Confluence/Jira via MCP (30 min) • A2A coordination demo: planner agent delegates to specialist sub-agents; trace the message flow (25 min) • Memory-aware agent: implement episodic recall on an IT support scenario (25 min) • LangGraph stateful workflow: multi-step document approval process with state persistence and checkpointing (35 min) • HITL approval flow: interrupt graph, capture human override decision, resume execution (25 min) • Context engineering exercise: optimise memory selection for a constrained context window (15 min)	Python 3.12+ LangGraph LangChain LangSmith (tracing) OpenAI SDK Anthropic Claude SDK MCP SDK Redis (episodic store) FAISS / ChromaDB Jupyter / VS Code

Module	Topics / Technical Coverage	Hrs	Hands-on	Tools
Module 5 **Cloud Deployment, Security & Governance, Evals & Group Exercise** **8 Hours**	Cloud Deployment Overview: Azure vs AWS strategy; Azure AI Foundry / Azure OpenAI Service; AWS Bedrock; model deployment options (overview only) Low-Code Agent Awareness: brief overview of Azure AI Foundry's low-code/no-code agent builder — contrasted with the code-first approach used in this course MLOps / LLMOps Fundamentals: model versioning, deployment pipelines, monitoring, cost management (overview) AI Risks: prompt injection, jailbreaks, tool abuse, data exfiltration vectors Guardrails Framework: input validation, output filtering, content safety layers, implementation patterns Tool Gating: permission models, capability scoping, audit trails Access Control & Data Safety: role-based, team-based, and geography-based / data-residency access control for AI/LLM systems AI Governance: policy frameworks, compliance and audit requirements, PII handling, data residency HITL Controls: confidence thresholds, escalation triggers, override logging Observability: LangSmith or Langfuse tracing, logging, latency and cost monitoring Evaluation Frameworks: Task Evaluation, Groundedness Evaluation, Safety Evaluation, Offline vs Online Evals Production Readiness Checklist Guided Group Exercise (see belw)	**8**	• Cloud deployment walkthrough: deploy a sample agent endpoint on Azure AI Foundry / Azure OpenAI — instructor-led demo (40 min) • Guardrails implementation: add input sanitisation + output validation + content filter to a provided agent scaffold (40 min) • Tool gating lab: define a permission model, simulate unauthorised tool call, verify block and audit log (30 min) • Access control lab: configure role/team/geography-based access policies for a multi-tenant agent scenario (35 min) • Safety testing: run adversarial prompt suite against the guardrailed agent, document pass/fail (30 min) • Observability lab: instrument an agent with LangSmith and Langfuse tracing, compare dashboards (35 min) • Build evaluation harness: task + groundedness + safety rubrics on the Phase 1 mini-project agent (40 min) • GUIDED GROUP EXERCISE (2 hrs): teams receive a pre-built agent scaffold for one scenario — (A) IT Incident Resolution Agent, (B) Document Intelligence Pipeline, (C) Knowledge Operations Assistant. Task: extend the scaffold to add RAG (LangChain or LlamaIndex), one memory layer, an HITL checkpoint, guardrails + access control, and run the eval harness with LangSmith/Langfuse tracing. Present findings (architecture decision, one key trade-off, eval result).	Azure AI Foundry / Azure OpenAI (demo) AWS Bedrock (overview) Python 3.12+ LangChain / LangGraph LangSmith Langfuse OpenAI SDK Anthropic Claude SDK Guardrails AI Pre-built agent scaffolds Eval harness template Access control policy template Group exercise brief Production checklis

Delivery Schedule

Day-by-Day Breakdown · Morning & Afternoon Sessions

Phase	Day	Morning (4 hrs)	Afternoon (4 hrs)
Phase 1 **8 hrs**	**Day 1** **8 hrs**	**Module 1 — GenAI · NLP Foundations** Generic OCR (Tesseract) demo, cloud OCR overview, LLM Architecture, Transformers, Embeddings incl. multilingual (Sentence Transformers)	**Module 1 (continued)** LLM behaviour exploration lab, OCR hands-on + cloud OCR comparison, multilingual embeddings mini-lab, Model selection exercise, Responsible AI (brief)
Phase 1 **8 hrs**	**Day 2** **8 hrs**	**Module 2 — Prompt Engineering (OpenAI + Claude)** Zero-shot · Few-shot · CoT · Structured outputs · OpenAI SDK function calling	**Module 2 (continued)** Claude SDK tool use, Agent Skills overview, OpenSpec walkthrough, LangChain agent build, Memory comparison, Mini-Project, Architecture review
Phase 2 **8 hrs**	**Day 3** **8 hrs**	**Module 3 — Enterprise RAG & Fine-Tuning** RAG fundamentals, Fine-tuning overview & decision framework, Vector DBs, Chunking strategies	**Module 3 (continued)** RAG build (LangChain + LlamaIndex), Enterprise connector lab, Enterprise use-case walkthrough, Retrieval eval, Fine-tuning decision workshop, Agentic RAG
Phase 2 **8 hrs**	**Day 4** **8 hrs**	**Module 4 — Multi-Agent Orchestration, Patterns & MCP** LangGraph fundamentals, ReAct/Reflection/Plan-Execute patterns, Pattern selection, Orchestration with dynamic model & prompt routing	**Module 4 (continued)** MCP architecture + enterprise connector demo (SharePoint/Confluence/Jira), A2A coordination, Memory architectures, LangGraph stateful workflow, HITL approval flow, Context engineering
Phase 2 **8 hrs**	**Day 5** **8 hrs**	**Module 5 — Cloud, Security & Governance** Cloud deployment overview (Azure AI Foundry/Azure OpenAI, AWS Bedrock), MLOps/LLMOps overview, Low-code awareness, AI risks, Guardrails, Tool gating	**Module 5 (continued)** Access control lab (role/team/geography), Safety testing, Observability (LangSmith + Langfuse), Eval harness build, Guided Group Exercise, Team readouts, Production readiness wrap-up

Prerequisites & Lab Environment

Participant Prerequisites

- Proficiency in Python programming (3.x) — the sole language prerequisite for this track.
- Familiarity with REST APIs and JSON data formats.
- Basic understanding of ML/AI concepts (not required to be deep).
- Experience with enterprise software architecture and systems integration.
- Note: Java/Spring AI is delivered as a separate track and is not a prerequisite for this course.

Lab Environment (Per Participant)

- Python 3.12+ · Jupyter Notebook or VS Code
- OpenAI API access (provided for bootcamp duration)
- Anthropic Claude API access (provided for bootcamp duration)
- LangChain · LangGraph · LangSmith (tracing) · Langfuse (tracing)
- LlamaIndex
- FAISS / ChromaDB (local vector store) · Redis (episodic memory)
- MCP SDK · Guardrails AI
- Tesseract OCR (local) · Cloud OCR service access for demo (Azure AI Vision or AWS Textract)
- Cloud environment access for demo: Azure AI Foundry / Azure OpenAI and/or AWS Bedrock (instructor-led demo; participant sandbox access to be finalised)
- Internet connectivity for API calls and package installation

Note: Final lab requirements — cloud sandbox access, and connector credentials for the SharePoint/Confluence/Jira demos — to be discussed and finalised with the client IT team.

Guided Group Exercise — Day 5 Scenarios

- (A) IT Incident Resolution Agent — extend scaffold to add RAG (LangChain/LlamaIndex), episodic memory, HITL checkpoint, guardrails + role-based access control, run eval harness with LangSmith/Langfuse tracing.
- (B) Document Intelligence Pipeline — extend scaffold to add chunked RAG on provided docs (incl. one enterprise connector), output validation, groundedness eval.
- (C) Knowledge Operations Assistant — extend scaffold to add A2A delegation, memory layer, safety filter + geography-based access policy, task eval.

Deliverables per team: 5-minute readout covering architecture decision, one key trade-off observed, and eval harness result.

© NIIT 2025-26 | All information within this document is intellectual property of NIIT (NIIT Ltd). No part of this document or programme design may be shared or used without the written permission of NIIT (NIIT Ltd).